

Programming Language Design

Module 13: Even More Monads

Slides © Grant Williams, [CC BY-SA 4.0](#).

This is an open educational resource.

Feel free to submit fixes, improvements, and new material [here](#).

Last module

We learned *more* monads.

Which ones did we learn? Can anyone list all the monads we know?

► [Click here to reveal](#)

This module

Monads are cool, they are each like mini-programming languages that have their own custom features.

Each time we make a new language, we define it in terms of a monad. A monad represents a statement, a command. When we create the `Applicative` and `Monad` instances, we define what it means to sequence two of these statements to achieve complex behavior.

But monads are all separate languages. They all have useful features, but they don't combine with each other.

In this module, we will learn about *Monad Transformers*, which are basically monad builders. We can use them to mix and match the features we want.

But first, let's combine reading and writing into one monad manually with `State` ...

What if I need reading and writing?

We can add write-only accumulators to the language with `Writer`. But we can't actually retrieve the value during the execution of the writer.

And we can add global read-only data with `Reader`.

But what if we want fully imperative code with a value that we can read *and* write.

This brings us to what is IMO the most complex built-in `Monad : State`.

Using this monad, we gain access to fully imperative code with some read/write state.

Does this require breaking the language? No. The state is treated in such a way that behind the scenes, everything is purely functional. However, we can pretend that there is read/write data.

First: why?

Let's imagine I have a programming language. It has this AST

```
data Expr = Ct | Cf | -- <- constant true and constant false values
           If { cond :: Expr, true :: Expr, false :: Expr } |
           While { cond :: Expr, body :: Expr } |
           Set { name :: String, value :: Expr } | ...--(strings? numbers? etc.)
```

Suppose I want to write the code to parse an `if` statement in my language. Maybe it looks something like:

```
if x == 20 then
  "it's 20"
else
  "it wasn't 20"
end
```

Writing the parser

To parse an `if` expression in our language, we need to find the keyword `if`, followed by an expression, followed by `then` followed by another expression, followed by `else`, followed by yet another expression, followed by `end`.

Here is a sketch of what it might look like (token constructors start with `T`):

```
parseIf :: [Token] -> (Expr, [Token]) -- returns the if and the remaining tokens
parseIf (TIf : rest) =
    let (condition, rest') = parseExpression rest
        (TThen : rest'') = rest'
        (trueExp, rest''') = parseExpression rest''
        (TElse : rest''''') = rest'''
        (falseExp, rest''''''') = parseExpression rest''''
        (TEnd : rest''''''''') = rest''''
    in (If condition trueExp falseExp, rest''''''') -- that's 6 ''''''s. Yikes!
```

Yikes!

I hope I don't have to say why this is suboptimal...

Yeah, the issue here is that every time we "change" the token stream by reading some tokens from it, we have to pass forward the modified version.

And heaven forbid you forget one of those "primes" (' s). Now you have a bug!

Luckily, we can use the `State` monad to fix this. A `State s a` is a program that is allowed to make use of an `s`, and then it will return an `a`. Internally, it looks like this:

```
data State s a = { runState :: s -> (a, s) }
```

It's a function that takes a state and returns a result value and a new state.

This is a little tough, so let's see the code first, and then we'll talk about how it works...

Getting tokens

First, let's write a little helper function to extract tokens and see `State` in action:

```
import Control.Monad.State.Lazy -- this is how we import the state monad
expectToken :: Token -> State [Token] ()
expectToken tok = do
    tokens <- get -- produces a monad whose result is its state
    if null tokens || head tokens /= tok
        then error $ "expected " ++ show tok
        else put $ tail tokens -- replaces the current state with its tail
```

This function takes a token that it will "expect" to be the next token. It returns a state program that will then "get" the current state (which is a list of tokens). Then, it either crashes the program with an `error`, or it replaces the state with its own tail (removing the token).

Understanding `get` and `put` are key to understanding `State`, so let's zoom in...

get and put detour

```
get :: State s s -- returns a State monad that just returns its own state
put :: s -> State s () -- produce a State monad with the given state
```

Here is some more intuition to help us understand monads in general and `State` in particular. Monads are containers that either store a result value or another computation that leads to a result value. There is always a result type.

The result type is the last part of a monad's type. So:

- an `IO Int` is an input-output program that returns an `Int`
- a `Maybe Float` is a program that, if it produces anything, will produce a `Float`
- a `Writer (Sum Int) String` is a program that writes to a `Sum Int` monoid, and then produces a `String`

get and put (2)

```
get :: State s s -- returns a State monad that just returns its own state
put :: s -> State s () -- produce a State monad with the given state
```

The first type argument of a `State` is the type of the state it stores. So `State String Int` is a program which stores a read/write `String`, and then returns an `Int`.

The `get` function therefore is a `State` monad whose result type is the same as its state type. It returns its own state so we can access it within another state.

The `put` function takes a state value of type `s`, and it produces a `State` which will use that value for its state. The resulting state does not return anything useful `()`.

Basic `get` and `put` example (3)

```
doubleIfEven :: State Int ()
doubleIfEven = do
  x <- get -- `get` produces a program which results in its state. Bind its result to x
  if x `mod` 2 == 0
    then put $ 2 * x
    else return () -- return () is a program which does not change its state
```

What is the state that we're getting? We don't know. `State` does not actually store a state. It simply has the ability to query a state that is given to it. In this way, it works the same way as `Reader`. `get` is like `ask`: query whatever value was passed to me.

```
main = print $ runState doubleIfEven 20 -- returns ((), 40)
```

In the above, we provide the starting state of `20` to `runState`. The program then `get`s it, so `x` is 20, and then replaces the state with `2 * x == 40`.

get / put Knowledge Check

1. Write `subtract1IfOdd` in the same fashion as the above. Test it, too.
2. Modify your `subtract1IfOdd` program so that not only does it subtract 1 from the state if the state is odd, but it also results in that value (i.e., returns it).
3. How does that change what gets printed when you `runState` ?

get / put KC answers

1.

```
subtractIfOdd :: State Int ()  
subtractIfOdd = do  
  x <- get  
  when (odd x) (put $ x - 1) -- let's use the nice 'when' function
```

2.

```
subtractIfOdd' :: State Int Int  
subtractIfOdd' = do  
  x <- get  
  when (odd x) (put $ x - 1)  
  get
```

3. We get `(6, 6)` instead of `(((), 6)`, because now there is a return value.

Combining states

Remember that monads, intuitively, are little programs.

We can combine two programs together with `>>` or by putting them on adjacent `do` lines.

So `subtract1ifOdd >> doubleIfEven` is a program that, first, if its state is odd, subtracts one from it, and then doubles its state (which is guaranteed to be even at that point).

Observe:

```
main = print $ runState (subtract1ifOdd >> doubleIfEven) 33 -- returns ((), 64)
```

We can also name this program:

```
subtract1ifOddThenDouble = do -- the type of this `do` is `State Int ()`  
  subtract1ifOdd  
  doubleIfEven
```

Questions?

Back to parsing

Now that we understand how to use state a little better, let's go back to the function we were working on to help us expect a token:

```
import Control.Monad.State.Lazy
expectToken :: Token -> State [Token] ()
expectToken tok = do
  tokens <- get -- produces a monad whose result is its state
  if null tokens || head tokens /= tok
    then error $ "expected " ++ show tok
    else put $ tail tokens -- replaces the current state with its tail
```

`expectToken` is a function that takes a token, then produces a program that will either pull that token from the token stream or crash with an error. (we'll learn later how to combine `Either` error handling with `State` so it doesn't have to crash)

Parsing with State (2)

```
parseIf' :: State [Token] Expr
parseIf' = do
  expectToken TIf -- use `expectToken` to make sure there's a TIf, and pop it
  condition <- parseExpression' -- this `State` returns an expression for us
  expectToken TThen
  trueExp <- parseExpression'
  expectToken TElse
  falseExp <- parseExpression'
  expectToken TEnd
  return $ If condition trueExp falseExp
```

Assuming we have a `parseExpression' :: State [Token] Expr` this is a nice, clean `State` program that will parse an `If` expression for us.

How would we write `parseExpression'` ?

Parsing with State (3)

```
parseExpression' :: State [Token] Expr
parseExpression' = do
  next <- head <$> get
  case next of
    TTrue  -> modify tail >> return Ct -- I only have true and false values
    TFalse -> modify tail >> return Cf -- right now.
    TIf    -> parseIf'
    other  -> error $ "parsing expression, unexpected token: " ++ show other
```

1. Remember that monads are also functors. `head <$> get` is a monad that returns the head of whatever `get` returned.
2. `modify` is a function that applies the given function to the state and replaces it.
`modify tail` is the same as `do { x <- get ; put $ tail x }`
3. We use `>>` to produce a new state program. First, modify, then return `Ct`.

State knowledge check 2

1. Define the modify function. Its type is `modify :: (s -> s) -> State s ()`
2. Suppose there was also a `TWhile` token and a `While { cond :: Expr, body :: Expr }` constructor. Define `parseWhile :: State [Token] Expr`.
3. Test out your code with a token stream. You can use `runState` to actually run a state program on a given state. It will return the resulting state and value. You'll also need to add `TWhile` and `TDo` tokens and modify `parseExpression'`.

State KC 2 answers

1.

```
modify f = do { x <- get ; put $ f x } -- or  
modify f = f <$> get >>= put
```

2.

```
parseWhile :: State [Token] Expr  
parseWhile = do  
  expectToken TWhile  
  condition <- parseExpression'  
  expectToken TDo  
  body <- parseExpression'  
  expectToken TEnd  
  return $ While condition body
```

State KC 2 answers (2)

3.

```
main = do
  print $ runState parseExpression'
    [TWhile,TTrue,TDot,TFalse,TEnd,TIf,TTrue,TThen,TFalse,TElse,TTrue,TEnd,TTrue,TFalse]
```

Results in (While {cond = Ct, body = Cf},
[TIf,TTrue,TThen,TFalse,TElse,TTrue,TEnd,TTrue,TFalse])

If I run (parseExpression' >> parseExpression'), I get
(If {cond = Ct, trueExp = Cf, falseExp = Ct},[TTrue,TFalse])

We first parsed the while-expression, then we threw away the result and parsed the if. We were left with a couple of random tokens in the stream that I put there afterward.

Practice: how would we avoid throwing away the results?

Questions?

How is state implemented?

State has the most complex monad implementation we've seen. Remember that a state is secretly implemented as a function from a state type `s` to a tuple `(a, s)`

Here is what it needs to do:

1. The simplest `State` is one that ignores its `s` and gives a constant value for `a`.
2. If I have two states, I combine them as follows:
 - i. First, we define a function that takes an `s` ...
 - ii. It will run the first state on it to get an `(a, s)`
 - iii. Then, run the second state on the output of the first state's `s` value
 - iv. The resulting function is what we return.

This is how `>>` works. `>>=` is similar, but the second state can depend on `a`.

The state `Monad` instance

```
instance Monad (State s) where
  return x = State $ (\s -> (x, s)) -- notice that this function preserves s
  s1 >>= f = State $ \s -> -- a State is always a function inside
    let (a1, s1') = runState s1 s -- execute state 1, get the new state
        s2 = f a1 -- use the function to produce a second state from result
    in runState s2 s1' -- run the second state with the result state
```

Frankly, when I was learning Haskell, I found this brutally hard to understand. I found it easier when I was already using state to go back and understand the bind definition. This is why we started with code rather than the `Monad` instance.

Fundamentally, the `>>=` must produce a new `State`. It defines a function, which takes an initial `s`, runs the left state on it, uses the result of that to generate another state, then executes that state on the intermediate state value.

I know what you're thinking...

No, I'm not going to test you on the `State` monad. It's just a little too hard to be fair at this point in your undergraduate career.

However, if you study it, it will help you. `State` is like a combination of `Reader` and `Writer`, and I am allowed to test you on those.

Remember that `Reader r a` is a `r -> a`

`Writer w a` is a `(a, w)`

So it makes sense that `State s a` is a `s -> (a, s)`

Questions?

Monads allow customizing the language

With monads, we have unlocked a lot of cool things. We can add new features like:

- System-level side effects and promises (IO)
- Early returns (Maybe)
- Exception handling (Either) (but in an orderly way: exceptions as values)
- Non-determinism (List)
- Dependency Injection (Reader)
- Logging and accumulation (Writer)
- Direct Imperative programming (State)

These are things that are awkward or impossible with pure function calls.

Using monads

Some of these monads have *combinators* that are designed to make the monad's features available:

- `print`, `putStr`, `getLine`, etc. for `IO`
- `ask` for `Reader`, `tell` for `Writer`
- `get`, `put`, and `modify` for `State`
- We can make our own for `Maybe`'s early returns: `abortIf c = when c Nothing`

Some combinators work for any monad, such as `when` and `forM`.

But there's a problem: we can't easily mix and match the monad-specific ones.

For example, what if I want to read some user input with `getLine`, but then I want to `tell` the result to a monoid?

What's missing?

Let's try it. Suppose I want both `tell` and `getLine` together. We'll define an instance of `IO` called `getNameAndLogIt`:

```
import System.IO --for hFlush
getNameAndLogIt :: IO ()
getNameAndLogIt = do
    putStr "Please enter your name: "
    hFlush stdout -- putStr doesn't flush automatically, unlike putStrLn
    name <- getLine
    putStrLn ""
```

This is fine by itself, we can use this `IO` program like this:

```
main = getNameAndLogIt
```

But it's not logging! How can we add logging functionality?

What's missing? (2)

```
import System.IO --for hFlush

getNameAndLogIt :: IO ()
getNameAndLogIt = do
    putStr "Please enter your name: "
    hFlush stdout -- putStr doesn't flush automatically, unlike putStrLn
    name <- getLine
    tell $ "They entered: " ++ name -- soo... can we just log here?
    putStrLn ""
```

Seems fine...what's wrong?

Remember, `tell :: Monoid w => w -> Writer w ()`

Also remember, in `do` notation, `do { a <- m ; f a }` means `m >>= f`

What is the type of `>>=` ?

What's missing? (3)

```
(>>=) :: Monad m => m a -> (a -> m b) -> m b
```

That is, for some monad `m`, *it gives the same monad out.*

`getLine` is an `IO`: `getLine :: IO String`.

`tell` is a `Writer`: `tell :: Monoid w => w -> Writer w ()`

So our `do` notation ends up trying to match this type:

```
IO String -> (w -> Writer w ()) -> IO ()
```

Which it can't, because `IO`'s bind doesn't know how to bind with a `Writer`!

Put another way: the `do` block is being used to build an `IO`. We can't just inject a `Writer` (with `tell`) inside it. `IO` doesn't automatically know how to work with `Writer`.

More fundamentally

When we create a monad, we're coming up with a little miniature language, with special statements. To do that, we need to decide what the "semicolons" mean (how to combine two statements into one statement).

Monads also represent computational pipelines that return something. That's why every monad we've seen has at least one type parameter (`IO Int` , `Maybe String` , `[Float]`) and sometimes more (`Writer String ()` , `Either Error Good` , `Reader Env Ret`)

That last type is the "return" value of the monad. That's why `return 7` works for any monad. It means "build this monad where there's a 7 in its 'return slot'". Every monad has a "return slot".

What's missing? (4)

But *in between* the monad instances, some kind of magic happens:

- In between two `Writers`, the monoids are `<>` d.
- In between two `IO` s, we combine them into one program.
- In between two `State` s, we build a new state function that first executes the first `State`, then passes the result to the second `State` and executes that.
- etc.

But what about in between a `Writer` and a `State`? What should happen? `State` doesn't have a monoid in it, and `Writer` doesn't take a state as input, it only writes to it.

The combination is not defined.

Questions

Monad Transformers

If we want to combine monads like that, we have to use a monad transformer.

Monad transformers add functionality to other monads. For example, if you wanted to add early returns to a writer, you could use `MaybeT (Writer String) ()`, which is a `Maybe` monad that also supports `tell`.

The `T` stands for `Transformer`. `MaybeT` is a type constructor that adds `Maybe` functionality to an existing `Monad`. In this case, `Writer`.

This is a common pattern in Haskell. There's also a `WriterT` to add logging to another monad. We can use transformers to layer on functionality. We'll talk about how `WriterT String Maybe ()` is different from `MaybeT (Writer String) ()` in a bit.

Almost every monad has a `T` variant to add its abilities to another monad. The only common one that doesn't is `IO`. `IO` doesn't have very interesting behavior.

Monad Transformers again

Monad transformers are a complex topic, and a little too advanced for me to feel comfortable requiring them. However, they are important for practical Haskell programming.

If you plan on actually building a real software system in Haskell, I've heard people say your first step is creating a monad transformer stack (I haven't tried real Haskell software engineering, but it does sound fun.)

This means using `type` or `newtype` to define a monad transformer that has all the features you need for the main logic of your app (the stuff directly called from `main`).

A monad transformer is a data type that will take a monad and "inject" new behavior.

Let's take a look at how we can use them to combine features from different monads.

MaybeT

One of the simplest monad transformers is one of the most useful: `MaybeT`.

`MaybeT` says "I want to add early returns to another monad."

For example, `MaybeT IO ()` is like an `IO ()`, but if any of the programs inside of it return a `Nothing`, the whole thing will quit.

`MaybeT` is a constructor. Its first argument is the monad type that we want to add the feature to. In this case `IO` is the monad type. The second argument is the return type of the new monad, which is going to be `()`.

Note, `MaybeT (IO ())` is wrong, because `(IO ())` has the wrong kind. The value you pass needs to be a Type constructor `* -> *`. `IO` is not a complete type, it's a type function. The monad we give to a monad transformers must be a type function. The `()` is actually the return type of the *new* monad, not part of the `IO`.

MaybeT example

Let's make a simple login routine to show this off. As always, let's be mindful of the fact that actual authentication will involve more steps than this. We're illustrating the value of early-returns, not expressing best practices for security (which this is *not* one).

```
ensureValidUser :: MaybeT IO ()
ensureValidUser = do
  lift $ putStr "enter your username: "
  lift $ hFlush stdout
  name <- lift getLine
  unless (name `elem` validUsers) $ do
    lift $ putStrLn "invalid user detected. exiting..."
    hoistMaybe Nothing
  where
    validUsers = ["alice", "bob", "camille"]
```

There are two things to cover first: `lift` and `hoistMaybe`. Let's do that!

lift

A monad transformer is always built "on top of" another monad. It's a new datatype.

The `MaybeT` data definition looks like this:

```
newtype MaybeT m a = MaybeT { runMaybeT :: m (Maybe a) }
```

`m` is the monad it takes. It stores a version of that monad which will return a `Maybe a` instead of an `a`.

That is, previously, the monad had an `a` in its return slot. Now it has a `Maybe a`. So, based on its results, we can bind it to another `MaybeT` if the return value was `Just`. But if it returned `Nothing`, we will ignore any further `MaybeT` we have to bind to.

lift (2)

The problem is that we want to use another monad, and add early returns to it.

So suppose we want to use `IO`. That means we want to use functions like `putStrLn`.

But `putStrLn :: String -> IO ()`, it's not a `String -> MaybeT IO ()`. So we can't bind an `IO ()` with a `MaybeT IO ()`. They are two different types!

`lift` takes an instance of the inner monad (the `IO a` in this case) and "wraps" it inside of the outer monad. So `lift` will convert an `IO a` into a `MaybeT IO a`.

It will also convert a `Writer String a` into a `MaybeT (Writer String) a`. Or a `Maybe a` into a `WriterT Maybe a`.

`lift` is a function of the `MonadTrans` typeclass. All monad transformers can "lift" monads (or other transformer stacks) into them. This typeclass also implies `Monad`.

lift for MaybeT

But if `lift` is part of the `MonadTrans` typeclass, that means every `MonadTrans` (which includes `MaybeT`) can have its own definition for it. What does `lift` do for `MaybeT`?

```
instance MonadTrans MaybeT where  
    lift = MaybeT . (fmap Just)
```

It just maps "Just" on it and puts the result inside a `MaybeT`.

So if the original `IO` returned `7`, now it will return `Just 7`, and that value will be stored inside the `MaybeT`.

lift for MaybeT (2)

This is what was meant by the type definition:

```
newtype MaybeT m a = MaybeT { runMaybeT :: m (Maybe a) }
```

Take a monad, only now it returns a `Maybe a`. So by default, make it `Just`.

Now let's revisit the original code ...

MaybeT example revisited

```
ensureValidUser :: MaybeT IO ()
ensureValidUser = do
  lift $ putStr "enter your username: "
  lift $ hFlush stdout
  name <- lift getLine
  unless (name `elem` validUsers) $ do
    lift $ putStrLn "invalid user detected. exiting..."
    hoistMaybe Nothing
  where
    validUsers = ["alice", "bob", "camille"]
```

Every time we want to do IO, we `lift` it to make it return a `Just IO`, which won't early return.

Eww...do we really have to write `lift` for *every single* inner monad we use?

Refactoring the lifts:

No, this is equivalent, and also more performant if this trivial optimization is turned off:

```
ensureValidUser' :: MaybeT IO ()
ensureValidUser' = do
  name <- lift $ do
    putStr "enter your username: "
    hFlush stdout
    getLine
  unless (name `elem` validUsers) $ do
    lift $ putStrLn "invalid user detected. exiting..."
    hoistMaybe Nothing
  where
    validUsers = ["alice", "bob", "camille"]
```

Like most of Haskell's features, `do` is an expression. Therefore, we can pass a `do` as the argument of `lift`. This `do` constructs an ordinary `IO`, which we then bind on the result of after converting it into an `IO (Maybe String)` using `lift`.

Refactoring the lifts:

It's actually a law of monad transformers that:

```
lift a >> lift b == lift (a >> b)
```

So it's possible the optimizer can recognize these opportunities.

"lifting" isn't free: we're slapping a "Just" on top of something. Not expensive, but not free, either. It's nice to only do this occasionally.

But there's still one function we haven't explained: `hoistMaybe` .

To understand it, consider this: what is the type of `Nothing` , and what type does it need to be`?

hoistMaybe

`Nothing` has type `Maybe a`. But we don't want it to be `Maybe`. We want it to be a `MaybeT m a`, where `m` is whatever monad we choose.

That's what `hoistMaybe` is. I can't find its exact source code (the Hackage page has been down for some time), but it probably looks like this:

```
hoistMaybe :: Monad m => Maybe a -> MaybeT m a
hoistMaybe Nothing = MaybeT (return Nothing)
hoistMaybe (Just x) = MaybeT (return (Just x))
```

That last line could also be `hoistMaybe (Just x) = lift $ return x`. Same result.

So now, we can convert a monad `m a` into a `MaybeT m a` with `lift`, and we can also convert a `Maybe a` into a `MaybeT m a` with `hoistMaybe`.

The complete program

```
ensureValidUser :: MaybeT IO ()
ensureValidUser = do
    name <- lift $ do
        putStr "enter your username: "
        hFlush stdout
        getLine
    unless (name `elem` validUsers) $ do
        lift $ putStrLn "invalid user detected. exiting..."
        hoistMaybe' Nothing
    where
        validUsers = ["alice", "bob", "camille"]

initializationRoutine :: MaybeT IO ()
initializationRoutine = do
    ensureValidUser
    lift $ putStrLn "user is valid, proceeding with initialization..."
-- vv we need to return () because runMaybeT returns a Maybe () instead of ()
main = do runMaybeT initializationRoutine ; return ()
```

How does `MaybeT` work?

We've seen that `MaybeT` is a `MonadTrans`, and that `Monad m => MonadTrans m`.

So that means `MaybeT` also has to be a `Monad`. It has to support `>>` and `>>=`.

What does `>>=` look like for `MaybeT`? Something like this:

```
instance Monad m => Monad (MaybeT m) where
  return x = MaybeT (return (just x))
  (MaybeT m) >>= f = MaybeT $ do
    result <- m
    case result of
      Nothing -> return Nothing
      Just a -> runMaybe $ f a
```

So `return` puts a monad that, given `x`, returns `Just x` inside the `MaybeT`.

And bind will either create an `m` that either returns `Nothing` or runs depending on `m`.

Questions?

Mixing and matching transformers

Okay, that's an example of using *one* transformer. But when are we ever going to stop with just one?

Let's take a look at combining `WriterT` *and* `MaybeT` together. Now we can log *and* early return.

Mixing `WriterT` and `MaybeT`

```
ensureValidUserLogging :: WriterT String (MaybeT IO) ()
ensureValidUserLogging = do
  name <- lift $ lift $ do -- we'll clean up lift $ lift later
    putStr "enter your username: "
    hFlush stdout
    getLine
  tell $ name ++ " attempted to log in...\n"
  unless (name `elem` validUsers) $ do
    lift $ lift $ putStrLn "invalid user detected. exiting..."
    tell "invalid user\n" -- we can "tell" without "lift": Writer is top
    lift $ hoistMaybe' Nothing -- now we "lift" to early return
  tell "valid user\n"
  where
    validUsers = ["alice", "bob", "camille"]
```

I know what you're thinking: "EW I do NOT like writing `lift $ lift`". Don't worry, we'll eliminate that later.

Mixing `WriterT` and `MaybeT` (2)

Now, let's see our new calling function and `main`:

```
initializationRoutineLogging :: WriterT String (MaybeT IO) ()
initializationRoutineLogging = do
    ensureValidUserLogging
    lift $ lift $ putStrLn "user is valid, proceeding with initialization..."
    tell "initializing\n"

main = do
    -- result is a Maybe((), String)
    result <- runMaybeT $ runWriterT initializationRoutineLogging
    case result of
        Just (res, log) -> putStrLn $ "log is: " ++ log
        Nothing -> putStrLn "we lost the log..."
```

We run our `runSomethingT` functions in reverse order. `runWriterT` takes a `WriterT`, so it goes first. It will return a `MaybeT`, which is why we call `runMaybeT` next.

Transformer order matters

Unfortunately, this result shows that something is wrong. We *want* the log!

Monad transformers create new monads that first, run the original monad (call the *inner monad*), and then lift the result into a new monad.

For example, `MaybeT` makes its inner monad return a `Maybe a` when it used to return an `a`. Its old result is lifted into a `Maybe`.

And `WriterT` makes its inner monad return a `(a, w)`, where `w` is a monoid.

That is, `newtype WriterT w m a = WriterT { runWriterT :: m (a, w) }`

The issue is, since `MaybeT` is the inner monad, when it is `Nothing`, it won't return anything. It won't keep the `w` log. It will lose it.

So the `Just` case keeps the log, but we want the log no matter what!

Fixing it

We can fix this by flipping the order. This is the wrong order:

```
ensureValidUserLogging :: WriterT String (MaybeT IO) ()
```

We really want this:

```
ensureValidUserLogging' :: MaybeT (WriterT String IO) ()
```

That is, instead of transforming a `Maybe` to return a `Writer`, we want to transform a `Writer` to return a `Maybe`. That way, we always have a monoid no matter what.

Remember, a writer returns an `(a, w)` where `w` is a monoid. So now it returns `(Maybe a, w)`. The `w` is always there.

Knowledge check: Can you rewrite our functions before to use this new transformer?

Knowledge check: transformers 1

```
ensureValidUserLogging' :: MaybeT (WriterT String IO) ()
ensureValidUserLogging' = do
  name <- lift $ lift $ do
    putStr "enter your username: "
    hFlush stdout
    getLine
  lift $ tell $ name ++ " attempted to log in...\n"
  unless (name `elem` validUsers) $ do
    lift $ lift $ putStrLn "invalid user detected. exiting..."
    tell "invalid user\n"
    hoistMaybe Nothing -- early return doesn't require a lift anymore
  lift $ tell "valid user\n"
where
  validUsers = ["alice", "bob", "camille"]
initializationRoutineLogging' :: MaybeT (WriterT String IO) ()
initializationRoutineLogging' = do
  ensureValidUserLogging'
  lift $ lift $ putStrLn "user is valid, proceeding with initialization..."
  tell "initializing\n"
```

Knowledge check: transformers 1, main

```
main :: IO ()
main = do
  -- maybeResult is a WriterT String (IO (Maybe ())).
  -- That is, it's a writer that always has its state available,
  -- but can optionally return another result.
  let maybeResult = runMaybeT initializationRoutineLogging'
  -- writerResult is an IO (Maybe(), String).
  let writerResult = runWriterT maybeResult
  -- now we can bind, because we're binding an IO in an IO.
  ioResult <- writerResult -- we can actually bind this one
  let (earlyReturnStatus, log) = ioResult
  -- we could have done runWriterT $ runMaybeT $ ... similar to before
  if isNothing earlyReturnStatus
    then putStrLn "initialization did not complete due to errors."
    else putStrLn "initialization was successfull..."
  -- we have the log no matter what now
  putStrLn "Authentication log:"
  putStrLn log
```


Questions?

I have a question!

Previously we had a `MaybeT (WriterT String IO) ()`, which was basically a `IO (Maybe (), String)` (after going through all the substitutions)

Why can't I just use that!? Just define an `IO (Maybe (), String)`. The `String` stores the log. The `Maybe ()` stores the early return.

Seems like we get everything we want! What's missing?

The binds are missing

The issue is, binding `IO` doesn't handle the log or early return.

Suppose we have this:

```
someEarlyReturningThing :: IO (Maybe ())
someLoggingThing :: IO String
someOtherLoggingThing :: IO String
someCombinedThing :: IO (Maybe (), String)
someCombinedThing = do
    result <- doTheEarlyReturningThing
    log1 <- someLoggingThing
    log2 <- someOtherLoggingThing
    return (result, log1 <> log2)
```

Notice: we didn't actually early return if `result` was value. `IO` does not ever early return. We also had to manually combine the logs with `<>`, instead of using `Writer` !

The real point of monad transformers

When we use monads, we don't just want a computation. Those are easy to represent as ordinary functions.

What we really want is some kind of "threaded effect". Some kind of feature that is associated with every statement in the combined monad.

We want these statements to automatically be combined together respecting the feature. Early returns short circuit the rest of the logic. Writers maintain a constant log. Readers maintain an environment. These features are "threaded in" between each statement.

Why monad transformers "modify" the inner monad

We transform monads by changing their return type. We don't just "hoist" the monad into a new environment.

`MaybeT (WriterT String IO) ()` is an `IO` which will have a monoid injected into its return type, which will then be wrapped in a `Maybe`.

Why not define the `T`'s backwards? `IOOf (WriterOf String Maybe) ()`

If we did it this way, an `IOOf` would have to have some universal way to run its inner monads. There happens to be one for `Writer` and `Maybe`, but it would mean nothing could "contain" an `IOOf`, because we can't safely run an `IO` and pull the result out.

Basically, it would only work for some monads.

Using the new transformer

Now it's time to address two things:

1. There is no way Haskell programmers, famously terse, actually write `lift $ lift $ lift $ ...` *ad infinitum* all the time
2. Do we seriously have to change everything around every time we want to change the order? Sometimes it makes sense to have `MaybeT (Writer ...)` and sometimes it's better to have a `WriterT ... Maybe` (when we want the early return to wipe out the log).

We can solve both these issues at the same time, and have much nicer, more flexible, less "lifty" code.

The monadic typeclasses

If you've been looking through the documentation for different monads, you've probably noticed that it's organized differently than in the slides.

For example, if you go look at the `Writer` documentation [here](#)...

You see that the top definition is not the `Writer` type. It's a typeclass called `MonadWriter`.

And if you keep scrolling down, you find it, but it's defined in terms of `WriterT` !

```
type Writer w = WriterT w Identity
```

What is `Identity`, and where did `Writer` go? It's just the Haskell equivalent of a `typedef`.

The monadic typeclasses (2)

The logic here is that we want `Writer` and `WriterT` to support the same operations.

We don't want to have to copy all the functions they need. It would be annoying if we used `tell` for `Writer`, but `tellT` for `WriterT`.

That would also make it so that if we refactored our code from a `MaybeT (Writer Log) ()` to a `WriterT Log Maybe ()`, we would have to change all the functions.

So instead, we define a typeclass that has all the useful combinators. All the `Writer` functions, such as `tell` are in the typeclass.

Then, we make both `Writer` and `WriterT` both be instances of this typeclass. This way, `tell` will work for both of them!

The Identity Monad

The identity monad is a monad that just represents a value `a`.

It's defined something like this: `data Identity a = Identity { runIdentity :: a }`

It doesn't do anything useful. It only exists so that when a higher-kinded type, such as a monad transformer, *needs* a monad as one of their parameters, you can say "actually, I don't want it to do anything".

This is why `Writer` is defined as `WriterT w Identity`. We've already gone through the trouble of defining `WriterT`, which has all the bind behavior in it and works on any inner monad.

So now, we just say "a `Writer` is when that inner monad doesn't do anything, it just returns the log". Now there's no special behavior. We end up with a plain `Writer`.

The monadic typeclasses (3)

This pattern repeats for almost every monad.

There is a typeclass whose name starts with `Monad...` that defines all the useful combinators, then a transformer (except for `IO` which has no transformer) and then the actual monad itself. If the transformer exists, the monad is usually defined as just `WhateverT (...other params...) Identity`.

And it turns out, we can use this pattern to dramatically simplify our code.

MonadIO

First, if you visit the [IO monad documentation](#), you'll see that there's a typeclass called `MonadIO`, which has a single combinator, `liftIO`.

This function just pulls an `IO` into any monad that implements `MonadIO`.

Why is that useful? Because pretty much every transformer implements `MonadIO`.

For example, from `Control.Monad.Trans`:

```
instance (MonadIO m) => MonadIO (WriterT w m) where
  liftIO = lift . liftIO
```

So anytime the inner monad `m` supports `liftIO`, the outer monad also supports it.

Therefore, we can do any number of `lift` s for `IO` all at once, with one `liftIO`.

Example of nicer code with `liftIO`

```
ensureValidUserLogging' :: MaybeT (WriterT String IO) ()
ensureValidUserLogging' = do
  name <- liftIO $ do
    putStr "enter your username: "
    hFlush stdout
    getLine
  lift $ tell $ name ++ " attempted to log in...\n"
  unless (name `elem` validUsers) $ do
    liftIO $ putStrLn "invalid user detected. exiting..."
    tell "invalid user\n"
    hoistMaybe Nothing -- early return doesn't require a lift anymore
  lift $ tell "valid user\n"
where
  validUsers = ["alice", "bob", "camille"]
```

Notice, no more `lift $ lift $ ...`, and so on. `liftIO` handles that for us. Because `WriterT ... IO` is a `MonadIO`, and `MaybeT (some MonadIO)` is also one.

But it's still not *that* nice...

There are still a lot of `lifts` in that code...

And it's still very finicky. If we swap the order of the transformers, it wouldn't work anymore because we'd have to remove a layer of lifts around some combinator calls, but add a new layer of lifts around others.

Luckily, we can use those fancy `MonadX` typeclasses to write truly flexible and *effectful* code.

Here's an example of a proper version of the previous functions...

Using the monadic typeclasses

```
ensureValidUserLogging'' :: (MonadWriter String m, MonadIO m, Alternative m) => m ()
ensureValidUserLogging'' = do
    name <- liftIO $ do
        putStr "enter your username: "
        hFlush stdout
        getLine
    tell $ name ++ " attempted to log in...\n"
    unless (name `elem` validUsers) $ do
        liftIO $ putStrLn "invalid user detected. exiting..."
        tell "invalid user\n"
        empty
    tell "valid user\n"
    where
        validUsers = ["alice", "bob", "camille"]

initializationRoutineLogging'' :: (MonadWriter String m, MonadIO m, Alternative m) => m ()
initializationRoutineLogging'' = do
    ensureValidUserLogging''
    liftIO $ putStrLn "user is valid, proceeding with initialization..."
    tell "initializing\n"
```

Some notes about that code

Notice that we're no longer writing a particular monad.

Instead, we're saying, "these actions can be embedded in any monad, as long as it is a `MonadIO`, a `MonadWriter`, and an `Alternative`."

`Alternative` is the typeclass that can handle early returns. The name represents the idea of "we can have one or more results, or maybe nothing at all". `MaybeT` and `Maybe` are both instances of `Alternative`. `empty` is the combinator that represents "early return" when we're using `MaybeT`.

(it's not called `MonadMaybe` because it also works with parsers)

So instead of creating a concrete monad stack. We're saying "here is a program of any monad, as long as that monad supports these features: early return, logging, and IO."

What about main?

Believe it or not, `main` doesn't need to change at all:

```
(earlyReturn, log) <- runWriterT $ runMaybeT initializationRoutineLogging''  
  
    if isNothing earlyReturn  
        then putStrLn "initialization did not complete due to errors."  
        else putStrLn "initialization was successfull..."  
  
    putStrLn "Authentication log:"  
    putStrLn log
```

Because we're saying `runWriterT $ runMaybeT ...`, Haskell's type inference knows that the return type of `initializationRoutineLogging''` needs to be `MaybeT (WriterT ... IO) ()`. It then further figures out the monoid is a `String`, because we're using `log` as a `String`.

What if we swapped the result?

What if we did this?

```
result <- runMaybeT $ runWriterT initializationRoutineLogging''
```

Notice, now it's `runMaybeT $ runWriterT ...` instead of `runWriterT $ runMaybeT ...`

Believe it or not, this works fine. Our function's type just said "I can convert these operations into any monad, as long as it supports early returns, logging, and IO".

`MaybeT (WriterT String IO) ()` and `WriterT String (MaybeT IO) ()` both work!

However, we still have the constraint that we identified before:

```
putStrLn $ case result of
  Nothing -> "initialization failed, but there's no log now!"
  Just (result, log) -> "Authentication log: " ++ log
```

We let the user decide

We did something cool just now without meaning to: we made it up to the user whether or not the log survives a failed login attempt.

Do we want the log? Probably. But the code will work either way, so the user can now decide what kind of return value they get. We have written a sub-routine in an *abstract programming language*, and let the user decide specifically how it gets executed!

Oh, one more thing. Let's alias all our constraints so we don't have to type them:

```
type AppMonad m = (MonadWriter String m, MonadIO m, Alternative m)
```

Now our main app control functions can be declared like:

```
initializationRoutine :: AppMonad m => m ()
```

Questions?

Nice exception handling

Remember when we learned that `Either e` is a monad, and it supports early returns which don't just return, but also carry an error?

Its monad transformer isn't called `EitherT` for some reason, but rather `ExceptT`. The new name makes sense: it adds exception handling support to a monad.

The typeclass is called `MonadError` instead of `MonadExcept` like you would expect. I'm not sure why this particular monad is so irregular in its naming.

Bad exception handling

Now, I'm not a fan of statement-based exception handling found in languages like C++ or Java. Getting a valid value out of a try block requires writing it to a variable scoped outside the block.

```
Something* the_value = nullptr;
try {
    the_value = something_that_might_fail();
} catch (Whatever const& exception) {
    the_value = DEFAULT_VALUE;
}
```

Yuck. Wouldn't it be nice if we could do this?

```
Something* the_value = try { something_that_might_fail() } or_else { DEFAULT };
```

Well, that's exceptions as values, which is what we're about to learn.

Bad exception handling (2)

The old fashioned "exception statement" approach to errors is reminiscent of even older techniques of setting jump handlers to trigger on an error. (See `setjmp` and `longjmp` in C).

It forces you to reason about control flow. If I throw an exception here, who will catch it? It might be ten frames up in the stack.

It also forces you to somehow know every error that could happen. In Java or C++, you can throw an exception at any time. If the caller does not catch it, control flow automatically jumps up to the next try block. That could be very far away.

Any function you call in Java or C++ (unless marked `noexcept`) can throw. It can happen at any time, and it's usually not part of the type. If it is part of the type, in Java, you *have* to wrap the function in a `try`. Yuck!

Good exception handling

So what do more modern languages do? They treat exceptions as "error" values. They don't get to hijack the program's control flow, they are just a normal value.

In Rust, there is a type called `result<T, E>`, where `T` is the type of the "good" value, and `E` is the type of the error. (In Rust, angle brackets go around type params)

Suppose you want to write a function that connects to a database, but returns an error string if it's not able. In (simplified) Rust, that might look like:

```
fn connect(host: Url) -> result<Connection, String> { ... }
```

So it doesn't return a `Connection`, it returns a result, which *could* be a valid connection, or it could be an error string.

Good exception handling (2)

But we *want* a `Connection`. We don't want a `result<Connection, String>`. What can we do with a `result`?

One thing we can do is "unwrap" it. This will pull the good value out if it exists, and panic (i.e, crash) otherwise: `let connection = connect(url).unwrap();`

If we want it to crash with a useful message, we can `expect` it instead:

```
let connection = connect(url).expect("couldn't connect")
```

But what if we want to do one set of things if the value is good, and one if the value is bad?

Good exception handling (3)

We can use `and_then` and `or_else` to run code that only runs in the "good" and "error" case.

```
let report_or_error: String =  
    connect(url)  
    .and_then(|db| { log!("connected to {}", url); Ok(db) } )  
    .and_then(|db| generate_report(db))  
    .unwrap_or_else(|err| {  
        let msg = format!("failed to connect to {}. Error: {}.", url, err);  
        log(msg); msg  
    })
```

(note: this is a sketch. In real Rust, you probably wouldn't want a single type representing both a report and an error. I don't want to introduce other Rust features right now, so we're keeping it simple.)

Good exception handling (4)

[Does that remind you of anything?]

And why is it good? It doesn't jump anywhere, and you're free to inspect error values without needing to create a whole try-catch block.

You also know what type the errors could be. An enum, a string, etc. You don't have to read the code to know what it's going to throw: it's included in the type.

Good exception handling (Rust and Haskell)

In Rust, `result<T, E>` is just `Either E T`.

It's the same, and it's a monad in both languages.

The `>>=` operator in Haskell is named `and_then` in Rust.

The `catchError` combinator (which we're about to talk about) is named `or_else` in Rust.

The only difference is that in `Haskell`, we don't have to write "and_then" over and over, because it happens automatically between two statements in a `do` block.

Monads are a common computational pattern. They show up in a lot of places. But in languages that recognize them with an explicit `Monad` typeclass or interface, there is the possibility for special syntax. This is why so many cool features are "monadic".

Let's use MonadError

For comparison, here's a way we can copy the Rust approach in Haskell:

```
type AppMonad' m = (MonadWriter String m, MonadError String m)
reportOrError :: Url -> AppMonad' m => m String
reportOrError url =
    do
        db <- liftEither $ connect url;
        tell $ "connected to " ++ url;
        return $ generateReport db
    `catchError` \err -> do
        let msg = "failed to connect to " ++ url ++ ". Error: " ++ err
        tell msg ; return msg
-- ^^^^ the part above is mostly equivalent to the Rust ^^^^
main = do
    (output, log) <- runWriterT $ runExceptT $ reportOrError "db.somedomain.com"
    putStrLn $ "output was: " ++ show output -- small diff: result is an Either
    putStrLn $ "log was: " ++ log
```

Questions?

A custom transformer

We've been learning to use existing transformers: each one adds a new feature.

But what if the feature we want doesn't exist?

I saw a cool feature appear in a bunch of more recent systems languages like [Zig](#) and [Odin](#). It's called the "defer" statement.

In languages with this feature, you can prefix a statement with "defer", and that statement will be executed at the *end* of the current block instead of right away:

```
// from Zig homepage (modified slightly)
var list: std.ArrayList(u32) = .empty;
const gpa = std.testing.allocator; // a memory allocator. In Zig they're explicit.
// Ensure the list is freed at scope exit.
defer list.deinit(gpa);
... // code comes after this, but list.deinit(gpa); runs at the very end!
```

Why defer?

This feature is useful because it's very common to do "request/release" pairs. For example, requesting some memory with `malloc`, and then `free` ing it. Or requesting a file handle with `fopen`, and then `fclose` ing it. Normally, you'd have to put the free at the very end of the block:

```
SomeStruct* s = malloc(sizeof(SomeStruct));  
// ... a bunch of code  
free(s); return result;
```

But there are some issues:

1. If we early-return, we have to remember to free each time we do so.
2. The free is as far away as possible from the malloc, making it harder to verify
3. The frees are technically supposed to go in reverse order. Easy to mix up.

Let's make a monad transformer!

First step, let's imagine the interface:

- `MonadIO` allows people to use `liftIO`
- `MonadWriter` allows people to use `tell` (and other combinators we won't cover)
- `MonadError` allows `throwError` and `catchError`

What would make sense to be in a `MonadDefer` ?

There's really only one thing

The only thing I can think of is `defer` :

```
class Monad m => MonadDefer m where  
  defer :: m () -> m ()
```

This combinator will take a monadic action that doesn't return anything, and "save" it to be executed later.

Ideally, if we call two defers in a row, we want them to execute in reverse order:

```
defer action1  
defer action2
```

should be equivalent to `action2 >> action1` .

A datatype and "runner"

We're making a monad transformer. It doesn't make sense to "defer" a statement unless there's a monadic statement to defer...

```
newtype DeferT m a = DeferT (m (a, m ()))  
    deriving Functor
```

The transformer will inject a dependent `m ()` to the return type of the original monad. This is the defer action. It doesn't return anything. This is the goal:

```
runDeferT :: Monad m => DeferT m a -> m a  
runDeferT (DeferT m) = do  
    (result, deferred) <- m -- *save* the original return value  
    deferred -- now run the deferred action (can be many >>'d together)  
    return result -- new action that returns the original result
```

The applicative instance

```
instance Monad m => Applicative (DeferT m) where
  (DeferT m_f) <*> (DeferT m_x) = DeferT $ do
    (f, d1) <- m_f
    (x, d2) <- m_x
    return (f x, d2 >> d1) -- sequence the deferrs in reverse order

  pure x = DeferT (pure (x, pure ())) -- default is no deferrals: "pure ()"
```

First, we run the monad inside the first `DeferT`. We get its function `f` and defers.

Then we run the second monad. We get its value `x` and its defers.

Then we produce a new `DeferT` that calls `f` on `x` but sequences the defers in opposite order.

The monad instance

```
instance Monad m => Monad (DeferT m) where
  return = pure
  (DeferT m_a) >>= f = DeferT $ do
    (a, d1) <- m_a
    let (DeferT m_b) = f a
    (b, d2) <- m_b
    return (b, d2 >> d1)
```

The monad instance is very similar to the `Applicative` instance, only we have to call `f` on the result of the monad inside first.

Once we have the result of `f`, we sequence it with the left `DeferT` just like how we did in the applicative.

The work so far

We've now created a transformer that takes a monad and adds a "defer action" to it.

When we sequence or bind two of these monads, we also create a new defer action by first running the second one's action and then the first.

But we still haven't created a convenient way to actually defer something yet. We need to actually implement a "defer" keyword.

Any ideas?

MonadDefer

Remember, a `MonadDefer` is any monad that supports the "defer" keyword.

```
class Monad m => MonadDefer m where
  defer :: m () -> m ()
```

Now let's make an instance for it:

```
instance Monad m => MonadDefer (DeferT m) where
  defer (DeferT m) =      -- vv this expression took a lot of brain juice
    DeferT $ return ((), m >>= snd)
  -- explanation: new defer is "execute m, then execute m's defer slot".
  -- equivalent to: DeferT $ return ((), runDeferT m )
```

To "defer" means to create a `MonadDefer` which returns the action as its "defer" action. Here, we use `>>=` to produce a new monad that does what `m` does and then executes its second result (which is the defer actions inside the defer statement).

What about the other typeclasses?

```
instance MonadTrans DeferT where
    lift m_a = DeferT $ (, return ()) <$> m_a
```

"Lifting" a monad into defer just means giving it an empty defer action.

Here, we're mapping the tuple constructor to whatever `m_a` returned, so now it's additionally returning an empty action (one that returns `()`).

```
instance MonadIO m => MonadIO (DeferT m) where
    liftIO = lift . liftIO
```

If the inner monad is capable of `IO`, `DeferT` should also be capable of `IO`. This prevents needing to write `lift $ lift $ lift $ lift ... putStrLn`.

The code means "first call `liftIO` on the inner monad, then lift the result into `DeferT`."

Does it work?

First, what do you expect this code to do?

```
deferTest :: (MonadDefer m, MonadIO m) => m ()
deferTest = do
    liftIO $ putStrLn "pretend we're opening a config file with db info"
    defer $ liftIO $ putStrLn "okay, close the file now."
    liftIO $ putStrLn "pretend we're opening a database connection"
    defer $ do
        defer $ liftIO $ putStrLn "defer inside defer?"
        liftIO $ putStrLn "close the connection (*before* the file!)"
    liftIO $ putStrLn "do some computation with the db..."

main = runDeferT deferTest
```

In your mind, try to guess what order the statements will actually get printed in.

The answer

It does work!

```
pretend we're opening a config file with db info
pretend we're opening a database connection
do some computation with the db...
close the connection (*before* the file!)
defer inside defer?
okay, close the file now.
```

Even the nested defer works. "defer inside defer?" is printed after we close the connection.

With this monad transformer, we can easily do whatever cleanup we need right away.

How does it interact with ExceptT?

When making monad transformers, it's important to consider how they interact with other monad transformers.

In this case, we probably want `ExceptT Err (DeferT ...) a` rather than `DeferT (ExceptT Err ...) a`

Why? What's the difference? [any thoughts?]

How does it interact with ExceptT? (2)

`ExceptT Err m a` means "modify the monad `m` so that it returns an `Either Err a` instead of just an `a` like it normally would.

`DeferT m a` means "modify the monad `m` so that it returns a `m (a, m ())` instead of just an `a`. The second of the pair (the `m ()`) is interpreted as the action to run after.

`DeferT (ExceptT Err ...) a` then means add a deferral mechanism to to the underlying `ExceptT` monad.

What's wrong with that? Because the defer result would only apply when the `Either` is `Right`. In the "error" case, the defer actions wouldn't apply. "Inner" monads are more fundamental, so we're saying "the deferall function is dependent on the error function"

That's probably not what we want: we typically want defer actions to *especially* apply in error cases. But maybe there are times when you don't. You have the option.

But does that even work?

```
deferTest2 :: (MonadDefer m, MonadIO m, MonadError String m) => m ()
deferTest2 = do
    liftIO $ putStrLn "pretend we're opening a file"
    defer $ liftIO $ putStrLn "okay, close the file"
    throwError "throw an error though"

main = do
    result <- runDeferT $ runExceptT deferTest2
    ...
```

Why won't this code run?

Something is still missing

In order for that code to work, this must be the type of the transformer stack:

```
ExceptT String (DeferT IO) ()
```

The problem is, `defer` only exists in the `MonadDefer` typeclass:

```
class Monad m => MonadDefer m where  
  defer :: m () -> m ()
```

So that means, `Except` does not support `defer` !

Are we doomed?

We're not doomed

We can just say "hey, if we wrap `except` around something that supports `defer`, then the `except` also supports `defer`."

```
instance MonadDefer m => MonadDefer (ExceptT s m) where
  defer :: MonadDefer m => ExceptT s m () -> ExceptT s m ()
  defer (ExceptT m) = lift $ defer $ void m
```

The logic is that if the monad inside the `ExceptT` supports `defer`, then run the action ignoring the result (what `void m` does) and lift that deferred action into an `ExceptT`.

This raises questions about what happens if an error happens inside some deferred code. This code here will throw away those `Left` values.

Could we fix this? Sure, but it would require changing the semantics of `defer`.

Working

```
deferTest2 :: (MonadDefer m, MonadIO m, MonadError String m) => m ()
deferTest2 = do
    liftIO $ putStrLn "pretend we're opening a file"
    defer $ liftIO $ putStrLn "okay, close the file"
    liftIO $ putStrLn "now operate on the file"
    throwError "throw an error though"
    defer $ liftIO $ putStrLn "this won't happen because we threw."
    liftIO $ putStrLn "more operations that won't run, but we still want to clean up!"
main = do
    result <- runDeferT $ runExceptT deferTest2
    print result
```

```
pretend we're opening a file
now operate on the file
okay, close the file
Left "throw an error though"
```

We threw an error but the file still got closed!

I still hate writing `liftIO`!

Understandable. This is a weakness in the prelude IO functions.

The prelude is just the set of modules and functions that are imported by default.

There are other preludes, and the language is customizable enough to use them.

This one replaces `putStrLn :: String -> IO ()` with `MonadIO m => putStrLn :: String -> m ()`.

So if you use `protolude`, you don't have to `liftIO`, as long as your transformer stack has a `MonadIO` in it.

But there is a deeper problem with the code we've seen...

The n-squared problem

We had to make a special instance of `MonadDefer` for `MonadErrors` whose inner monads supported `MonadDefer`.

If we want a transformer stack to be `MonadDefer` when the top transformer is not `DeferT`, then we have to have every transformer above `DeferT` be `MonadDefer`.

Likewise, if we want a transformer stack to be `MonadIO`, we need every transformer above `IO` to also be `MonadIO`. Frequently this is all of them.

In fact, every time we add a new transformer, we often need to then implement `MonadWhatever` on every *other* transformer.

This means that if we have `n` transformers, we actually need up to `n^2` instances to be defined. I've seen this called the "n-squared instances problem" for this reason. There is a solution...

Questions?

Algebraic effects

Okay, for real this time, you are not required to learn this and I haven't gone and implemented a language feature in it.

But there is a strategy to fixing the n-squared instances problem that is exciting and has been catching on in many other functional programming languages ([OCaml](#) and [FSharp](#) in particular)

It's called *Algebraic Effects*. [Haskell](#) supports it too.

Algebraic effects (2)

The basic idea is that instead of creating a monad transformer, you create a data type that encodes the operations you are interested in. For a basic IO effect, we could create a type like [this](#):

```
data Teletype (m :: Type -> Type) k where
  Read  :: Teletype m String
  Write :: String -> Teletype m ()
```

Those are special commands that will become available to use inside of any monad `m` that has the `Teletype` effect in its signature `sig`.

(note, `data ... where` is using an advanced feature called [Generalized Algebraic Data Types](#) (GADTs)--it lets us give constructors more precise types so we don't have to pattern match everywhere)

Algebraic effects (3)

Somewhere else, you define an `Algebra` that is basically an interpreter for these custom messages.

```
newtype TeletypeIOC m a = TeletypeIOC { runTeletypeIO :: m a }
    deriving (Applicative, Functor, Monad, MonadIO)

instance (MonadIO m, Algebra sig m) => Algebra (Teletype :+: sig) (TeletypeIOC m) where
    alg hdl sig ctx = case sig of
        L Read      -> (<$ ctx) <$> liftIO getLine
        L (Write s) -> ctx      <$ liftIO (putStrLn s)
        R other     -> TeletypeIOC (alg (runTeletypeIO . hdl) other ctx)
```

Don't worry too much about understanding this. Focus on the fact that this algebra can be "injected" into a set of types (the `:+:` is a type constructor for these) and it does the operations it recognizes. It then leaves other operations to other algebras (the `R other` part).

Algebraic effects (4)

The "algebraic" part is that we can compose effects. We can use them like the constraints on our monads before.

What's cool though is that we only require one of those algebras per effect. We don't need n^2 .

Why?

Writing with algebraic effects is like defining a custom language that exactly matches your problem space. I've heard it called a "custom alphabet" (using a state machine metaphor).

Think of it as like a type-safe, debuggable, custom scripting language.

A domain-specific language.

This seems like a powerful ability and it is. What is its competitor?

Comparison with Macros

One powerful way to add features to a language is "macros".

We talked about them briefly when learning Lisp. A macro is a function that takes an abstract syntax tree and returns one.

You can also use them to add new features to a language. Like this macro I use in the Fennel programming language to add "incrementing" to the language:

```
(macro inc! [x] `(set ,x (+ 1 ,x)))
```

It makes it so that whenever you write `inc!`, the token to its right is interpreted as a variable. We end up generating the code `(set varName (+ 1 varName))`, which is incrementing.

Comparison with Macros (2)

Macros are actually a bit more powerful individually than monads or effects. They let you write code that tells the compiler how to generate code. (The fused effects library uses macros in Haskell)

In some ways, this is more powerful. We can actually customize raw syntactic rules to inject a completely custom programming language into our language.

However, it's less composable. I couldn't necessarily combine the features of two such custom languages.

Monads, through the transformers or effects, are highly composable. We can add new features to our stack by inserting another transformer layer or adding another type constraint.

Questions?

Finishing up

We've covered a truly enormous amount of material this semester.

Haskell has likely made you aware of just how real brain-fatigue is. It certainly did me!

But there are a few takeaways I want you to have that I hope have come through...

Some takeaways

Programming languages can start with what is easy to compute. "We know we have to generate assembly or intermediate code. What are the most advanced features we can handle?"

But they can also start from the other end. "We know what is possible to express mathematically. How much of that can be turned into a runnable computation?"

Haskell is likely the first language you learned of the second type.

By starting with math, we unlock a kind of "idea purity" that is very interesting. We abstract things so much that everything we're left with is essential.

A monad is kind of the "essential" idea of a dynamic computational structure. A primitive kind of program which can be executed. And an effect is something that the abstract computer can do.

Some takeaways (2)

Since a monad is the "essence" of data-dependent sequential computation, any time you need that, you're going to use/make a monad (or effect), intentionally or not:

- Your event scripting system in your video game is probably monadic
- The logic driving your dynamic music visualizer is probably monadic
- Your async logic that allows you to encode threads of computation in a promise in Javascript is monadic (and sometimes is literally a monad: it's just `IO` in Haskell).
- Your custom logging system is probably monadic. Or at least, it *could* be one.
- Your configuration system that makes the global configuration available as a singleton is monadic (reader)
- Your exception system is probably monadic.

If you know the theory, you can use it deliberately instead of haphazardly.

Some takeaways (3)

Because of the fact that so many things are monads, you can either understand what a monad is conceptually and write your system to benefit from the theory behind them...

...or you don't. And you end up with Java's exception system. Or Java's singleton classes. Or Java's initial threading system in which every object was implicitly a monitor.

What's wrong with Java? Bad theory meant that you have to interact with a bunch of random details to use its features, because the "essence" of ideas was not discovered. For example: there's no generic syntax to sequence Streams, Optionals, and Futures, because they didn't know about monads (i.e., there's no `do` notation or HKTs).

Basically, with incomplete theory, you end up with Java v. 1.0! And nobody wants that.

(they've honestly fixed a lot of Java by incorporating these higher-math ideas, so another valuable lesson is that no system is impossible to clean up!)

Conclusion

There will be more math developed and more advanced programming language features.

Remember the blub paradox. We are always blub programmers. Haskell is blub!

Never stop learning!